Bot Lifecycle Safety Gates

A friendly engineering briefing on how our trading bots should start, stop, reconcile, and share one broker account safely.

Personal consumption draft - June 27, 2026

The mechanical picture

Treat the broker account like a shared machine bed: many controllers can request motion, but only one actuator controller should move it.

The failure we must fix first

On watchdog lease loss, do not disconnect until flatten/cancel is proven or unresolved exposure is durably recorded.

The target design

AccountOwner becomes the single broker writer for one account. Bots send it intents; it owns actuation and reconciliation.

The operator view

A gate board shows what is pass, block, poison, or freeze, and every row comes from the same predicate that enforces it.

Plain-English promise

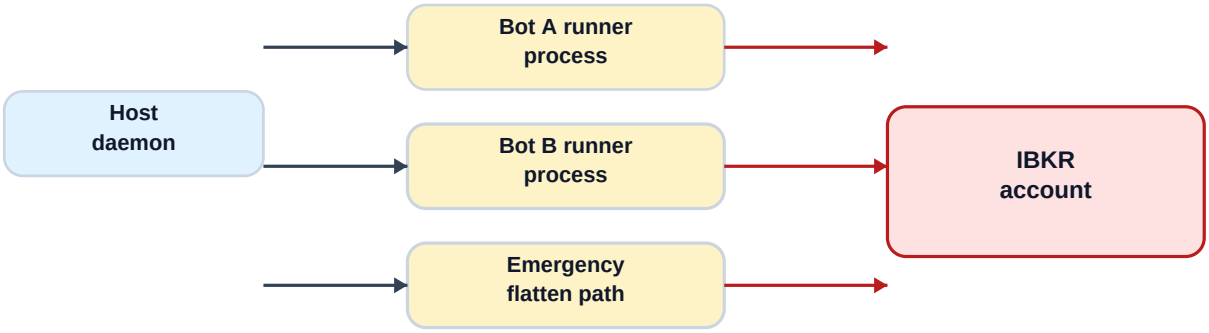
This document is not meant to be a software spec you have to decode. It is the system safety picture: what can move the account, what gates stop unsafe motion, and what we should build first.

1. The One-Sentence Problem

The IBKR account is a shared physical resource. If several bot processes can each send orders to it, then a process that is stale, crashed, or only half-aware can still change the account.

In mechanical terms: several control panels are wired to the same actuator. Local interlocks inside each panel help, but they do not create one system-wide interlock.

Today: several runner processes can write to the same account



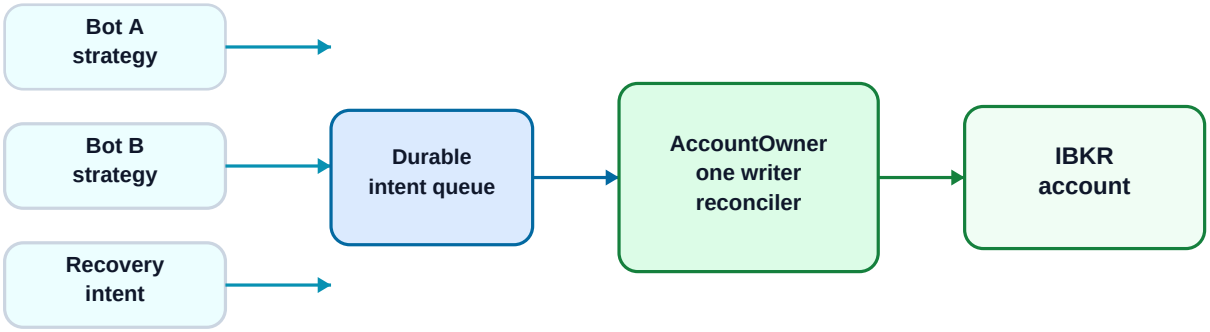
Risk: each runner has only a local lock, so account-level writes are not owned by one controller.

What exists today	Why it helps	Why it is not enough
One low-level `placeOrder` call site	Good choke point for IBKR order placement.	Multiple runner processes can still reach that chokepoint.
Per-runner submit lock	Prevents one runner from racing itself.	It does not serialize Bot A vs Bot B.
Cold-start reconciliation	Finds broker residue before a runner trades.	Current shape is mostly per-instance, not account-owner level.
Watchdog shutdown	Tries to pause and flatten on lease loss.	Today it can disconnect after unproven flatten, losing proof at the worst time.

2. Target Architecture: One AccountOwner

For a shared account, the safe design is not "every bot gets a better lock." The safe design is one account-level owner that is the only process allowed to talk to the broker.

Target: one AccountOwner writes to the broker account



Safety idea: runners can think, but only AccountOwner actuates the shared account.

How to read this:

Part	Friendly meaning	Safety job
Bot runner	The strategy brain. It watches bars and decides what it wants.	It emits an intent, but does not move the account directly.
Durable intent queue	The work order clipboard.	If something crashes, we still know what was requested.
AccountOwner	The account actuator controller.	It serializes orders, stamps ownership, resolves uncertain submits, and reconciles the whole account.
IBKR account	The shared machine bed.	Only AccountOwner is wired to move it in shared-account mode.

This is the key design decision: current runtime is R2, but the PRD commits shared accounts to R3. R3 means one writer process, not many writers plus a cross-process lock.

3. The Gate Board: A Dashboard of Interlocks

Think of the gate board like a machine status panel. Each row is an interlock. It says whether the system can deploy, start, reconcile, activate, submit, or recover.

Most important rule: the status panel must use the same predicate as the enforcement point. If the row says pass, the action must pass for that same observation. No parallel guesswork.

Gate phase	Question it answers	If it fails
Deploy / account preflight	Is this account clean enough to put another bot into?	Block deploy/start before we spawn a process.
Start recheck	Has the run been poisoned, stopped, or already started?	Refuse stale UI clicks.
Reconcile	Does broker reality match our durable book?	Continue, adopt, poison the run, or freeze the account.
Activate	Can this instance enter the live bar loop?	Stay blocked or paused.
Submit	Can this intent safely become a broker order?	Refuse, retry/adopt, or freeze if not provable.
Recovery	Can we prove the account is safe after a failure?	Keep account frozen or require audited operator override.

Pass
The interlock is satisfied.

Block
Action is refused, but the account is not necessarily unsafe.

Poison
This run is unsafe and must be redeployed.

Freeze
The whole account is unsafe for new deploys until recovery proves it clean.

4. The Immediate Fix: Disconnect Last

The highest-priority live bug is shutdown ordering after the daemon lease is lost. The safe rule is simple: keep broker visibility until we have proof, or until we have durable unresolved evidence.

Case	Safe result
Flatten completes and broker probe proves flat/no open orders	Safe incident outcome; run may exit or poison itself without contaminating the account.
Flatten times out, broker still reachable but state is not clean	Persist unresolved incident; freeze account when account artifact exists.
Broker unreachable, so proof is impossible	Persist unresolved incident; allow audited operator override if operator confirms flat externally.

Mechanical analogy: do not power down the sensor package before confirming the actuator reached a safe position. If you cannot confirm, tag the machine out.

5. Account Safety Artifacts

These are the durable records that let the system distinguish "this is known and safe" from "we are guessing." They are boring on purpose. Boring records are what make recovery possible.

Artifact	Plain-English job	Why it matters
instance_registry.jsonl	Append-only roster of bots and namespaces for an account.	If a broker order has a namespace, we can tell whether it belongs to any known bot.
account_baseline.json	Operator-approved reset point after verifying flat/no open orders.	Lets old completed residue be ignored only under explicit conditions.
unresolved_exposure.flag	Account tag-out flag.	Blocks deploy/start/submit when exposure cannot be proven safe.
operator_override.jsonl	Manual recovery log.	Prevents deadlock when IBKR is unreachable but the operator verified flat state externally.
account_events.jsonl	Account-level black box recorder.	Captures freeze, unfreeze, baseline, restart intensity, and AccountOwner events.

Critical point: the registry is safety equipment, not decoration. It must be written before first submit and cannot be rebuilt only from run directories.

6. Classification: Stop Thinking Only Per Bot

The old mental model asks, "Is this broker artifact mine?" In a shared account, that is too narrow. The safer question is, "Is this broker artifact accounted for by the account owner?"

Broker finding	AccountOwner classification
Matches expected live intent	Continue.
Owned orphan with recognizable namespace	Adopt and record it. Pause if exposure is ambiguous.
Owned by a known active sibling	Accounted for, not this bot contamination.
Owned by a poisoned/dead sibling with residue	Not ignored. Route to account recovery or freeze.
Unowned completed execution before approved baseline	Ignore only if account flat, no open orders, timestamp allowed, and id listed.
Unowned open order	Never baselineable. Freeze or poison path.
Unowned live position	Freeze until flattened/reconciled or audited override.
Probe failure or unparseable reference	Fail closed. Freeze if account safety cannot be proven.

This avoids the false comfort of "sibling-known means ignore." If the sibling is dead and nobody owns the cleanup, the account still has a hazard.

7. Build Order and Priorities

The implementation plan is sequenced to fix the active exposure leak first, then make the account model durable, then migrate to the single-writer architecture.

Slice	Build	Done when
P0	Design lock: current system is multi-process R2; shared accounts target R3 AccountOwner.	No cross-process-lock-only plan remains.
P1	Watchdog proof-before-disconnect.	Lease-loss proof failure persists unresolved evidence before disconnect.
P2	Single-source gate board.	Displayed gate status matches enforcement in tests.
P3	Account artifacts.	Freeze blocks start/deploy; registry is write-ahead.
P4	Account-scoped classifier.	Dead sibling residue is not blindly ignored; unowned open order freezes.
P5	AccountOwner MVP.	Runner processes cannot call IBKR placeOrder in shared-account mode.
P6	Fleet supervisor.	Restart intensity, baseline workflow, reconnect drain, and account gates are visible.

First fix
Watchdog disconnect-last. Smallest change, biggest live safety impact.

Then visibility
Gate board from enforcement predicates, so the panel cannot lie.

Then account memory
Registry, baseline, freeze, override.

Then R3
AccountOwner becomes the only broker writer.

8. Tiny Glossary

Term	Friendly meaning
Bot runner	The process that runs one strategy and decides what it wants to do.
AccountOwner	The one process allowed to submit broker orders for a shared account.
Intent	A durable request from a bot runner, like "buy one share" or "flatten."
Gate	An interlock that must pass before an action proceeds.
Poisoned run	This specific run is unsafe to restart. Redeploy is required.
Frozen account	The shared account is tagged out until recovery proves it safe.
Baseline	An explicit operator-approved reset point after verifying the account is clean.
Override	Audited manual recovery when automated proof is impossible, usually because IBKR is unreachable.
Reconcile	Compare broker reality against our durable book and classify every difference.

Bottom line: the safe architecture is not just more checks. It is one writer per account, visible interlocks, durable evidence, and a recovery path that never disconnects before proof or quarantine.